

A Simple Example: HelloWorld

This example shows you how to use JAX-RPC to create a Web service named `HelloWorld`. A remote client of the `HelloWorld` service can invoke the `sayHello` method, which accepts a string parameter and then returns a string.

HelloWorld at Runtime

Figure 9-1 shows a simplified view of the `HelloWorld` service after it's been deployed. Here's a more detailed description of what happens at runtime:

1. To call a remote procedure, the `HelloClient` program invokes a method on a stub, a local object that represents the remote service.
2. The stub invokes routines in the JAX-RPC runtime system.
3. The runtime system converts the remote method call into a SOAP message and then transmits the message as an HTTP request.
4. When the server receives the HTTP request, the JAX-RPC runtime system extracts the SOAP message from the request and translates it into a method call.
5. The JAX-RPC runtime system invokes the method on the tie object.
6. The tie object invokes the method on the implementation of the `HelloWorld` service.
7. The runtime system on the server converts the method's response into a SOAP message and then transmits the message back to the client as an HTTP response.
8. On the client, the JAX-RPC runtime system extracts the SOAP message from the HTTP response and then translates it into a method response for the `HelloClient` program.

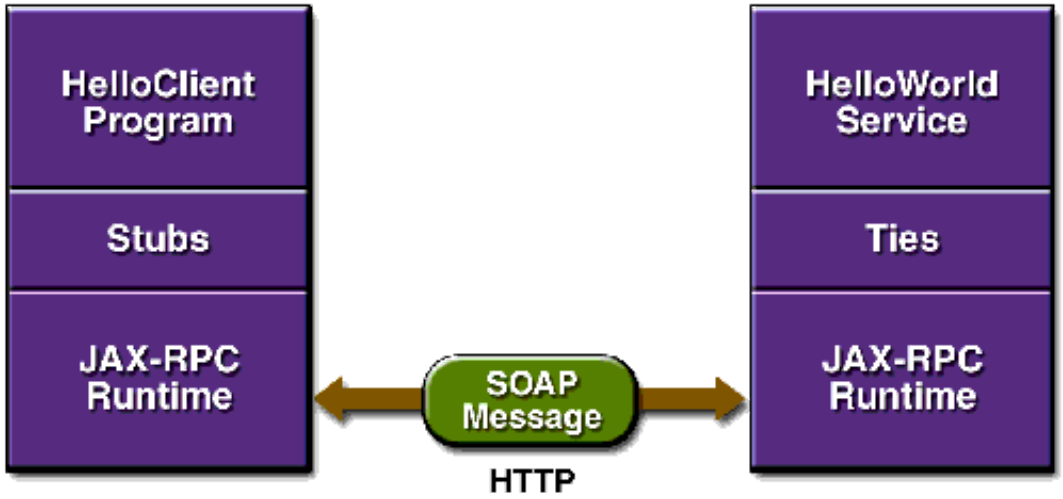


Figure 9-1 The `HelloWorld` Example at Runtime

The application developer only provides the top layers in the stacks depicted by Figure 9-1. Table 9-1 shows where the layers originate.

Table 9-1 Who (or What) Provides the Layers

Layer	Source
`HelloClient` Program `HelloWorld` Service (definition interface and implementation class)	Provided by the application developer
Stubs	Generated by the `wscompile` tool, which is run by the application developer
Ties	Generated by the `wsdeploy` tool, which is run by the application developer
JAX-RPC Runtime System	Included with the Java WSDP

HelloWorld Files

To create a service with JAX-RPC, an application developer needs to provide a few files. For the `HelloWorld` example, these files are in the *<JWSDP_HOME>*`/docs/tutorial/examples/jaxrpc/hello` directory:

- `HelloIF.java` - the service definition interface
- `HelloImpl.java` - the service definition implementation class, it implements the `HelloIF` interface
- `HelloClient.java` - the remote client that contacts the service and then invokes the `sayHello` method
- `config.xml` - a configuration file read by the `wscompile` tool
- `jaxrpc-ri.xml` - a configuration file read by the `wsdeploy` tool
- `web.xml` - a deployment descriptor for the Web component (a servlet) that dispatches to the service

Setting Up

If you haven't already done so, follow these instructions in the chapter Getting Started With Tomcat:

- Setting the PATH Variable
- Creating the Build Properties File
- Starting Tomcat

Building and Deploying the Service

The basic steps for developing a JAX-RPC Web service are as follows.

1. Code the service definition interface and implementation class.
2. Compile the service definition code of step 1.
3. Package the code in a WAR file.
4. Generate the ties and the WSDL file.
5. Deploy the service.

The sections that follow describe each of these steps in more detail.

Coding the Service Definition Interface and Implementation Class

A service definition interface declares the methods that a remote client may invoke on the service. The interface must conform to a few rules:

- It extends the `java.rmi.Remote` interface.

- It must not have constant declarations, such as `public final static`.
- The methods must throw the `java.rmi.RemoteException` or one of its subclasses. (The methods may also throw service-specific exceptions.)
- Method parameters and return types must be supported JAX-RPC types. See the section Types Supported By JAX-RPC.

In this example, the service definition interface is `HelloIF.java`:

```
package hello;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HelloIF extends Remote {
    public String sayHello(String s) throws RemoteException;
}
```

In addition to the interface, you'll need to code the class that implements the interface. In this example, the implementation class is called `HelloImpl`:

```
package hello;

public class HelloImpl implements HelloIF {

    public String message ="Hello";

    public String sayHello(String s) {
        return message + s;
    }
}
```

Compiling the Service Definition Code

To compile `HelloIF.java` and `HelloImpl.java`, go to the *<JWSDP_HOME>*`/docs/tutorial/examples/jaxrpc/hello` directory and type the following:

```
ant compile-server
```

This command places the resulting class files in the `build/shared` subdirectory.

Packaging the WAR File

To create the WAR file that contains the service code, type these commands:

```
ant setup-web-inf
ant package
```

The `setup-web-inf` target copies the class and XML files to the `build/WEB-INF` subdirectory. The `package` target runs the `jar` command and bundles the files into a WAR file named `dist/hello-portable.war`. This WAR file is not ready for deployment because it does not contain the tie classes. You'll learn how to create a deployable WAR file in the next section. The `hello-portable.war` contains the following files:

```
WEB-INF/classes/hello/HelloIF.class
WEB-INF/classes/hello/HelloImpl.class
WEB-INF/jaxrpc-ri.xml
WEB-INF/web.xml
```

The class files were created by the `compile-server` target shown in the previous section. The `web.xml` file is the deployment descriptor for the Web application that implements the service. Unlike the `web.xml` file, the `jaxrpc-ri.xml` file is not part of the specifications and is implementation-specific. The `jaxrpc-ri.xml` file for this example follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<webServices
    xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/dd"
    version="1.0"
    targetNamespaceBase="http://com.test/wsdl"
    typeNamespaceBase="http://com.test/types"
    urlPatternBase="/ws">

    <endpoint
        name="MyHello"
        displayName="HelloWorld Service"
        description="A simple web service"
        interface="hello.HelloIF"
        implementation="hello.HelloImpl"/>

    <endpointMapping
        endpointName="MyHello"
        urlPattern="/hello"/>
```

```
</webServices>
```

Several of the `webServices` attributes, such as `targetNamespaceBase`, are used in the WSDL file, which you'll create in the next section. (WSDL files can be complex and are not discussed in this tutorial. See Further Information.) Note that the `urlPattern` value (`/hello`) is part of the service's URL, which is described in the section Verifying the Deployment).

For more information about the syntax of the `jaxrpc-ri.xml` file, see the XML Schema file: `<JWSDP_HOME>/docs/tutorial/examples/jaxrpc/common/jax-rpc-ri-dd.xsd`.

Generating the Ties and the WSDL File

To generate the ties and the WSDL file, type the following:

```
ant process-war
```

This command runs the `wsdeploy` tool as follows:

```
wsdeploy -tmpdir build/wsdeploy-generated
-o dist/hello-deployable.war dist/hello-portable.war
```

This command runs the `wsdeploy` tool, which performs these tasks:

- Reads the `dist/hello-portable.war` file as input
- Gets information from the `jaxrpc-ri.xml` file that's inside the `hello-portable.war` file
- Generates the tie classes for the service
- Generates a WSDL file named `MyHello.wsdl`
- Packages the tie classes, the `Hello.wsdl` file, and the contents of `hello-portable.war` file into a deployable WAR file named `dist/hello-jaxrpc.war`

The `-tmpdir` option specifies the directory where `wsdeploy` stores the files that it generates, including the WSDL file, tie classes, and intermediate source code files. If you specify the `-keep` option, these files are not deleted.

There are several ways to access the WSDL file generated by `wsdeploy`:

- Run `wsdeploy` with the `-keep` option and locate the WSDL file in the directory specified by the `-tmpdir` option.
- Unpack (`jar -x`) the WAR file output by `wsdeploy` and locate the WSDL file in the `WEB-INF` directory.

- Deploy and verify the service as described in the following sections. A link to the WSDL file is on the HTML page of the URL shown in Verifying the Deployment.

Note that the `wsdeploy` tool does not deploy the service; instead, it creates a WAR file that is ready for deployment. In the next section, you will deploy the service in the `hello-jaxrpc.war` file that was created by `wsdeploy`.

Deploying the Service

To deploy the service, type the following:

```
ant deploy
```

For subsequent deployments , run `ant redeploy` as described in the section Iterative Development.

Verifying the Deployment

To verify that the service has been successfully deployed, open a browser window and specify the service endpoint's URL:

```
http://localhost:8080/hello-jaxrpc/hello
```

The browser should display a page titled Web Services, which lists the port name `MyHello` with a status of `ACTIVE`. This page also has a URL to the service's WSDL file.

The `hello-jaxrpc` portion of the URL is the context path of the servlet that implements the `HelloWorld` service. This portion corresponds to the prefix of the `hello-jaxrpc.war` file. The `/hello` string of the URL matches the value of the `urlPattern` attribute of the `jaxrpc-ri.xml` file. Note that the forward slash in the `/hello` value of `urlPattern` is required. For a full listing of the `jaxrpc-ri.xml` file, see Packaging the WAR File.

Undeploying the Service

At this point in the tutorial, do not undeploy the service. When you are finished with this example, you can undeploy the service by typing this command:

```
ant undeploy
```

Building and Running the Client

To develop a JAX-RPC client, you follow these steps:

1. Generate the stubs.
2. Code the client.
3. Compile the client code.
4. Package the client classes into a JAR file.
5. Run the client.

The following sections describe each of these steps.

Generating the Stubs

Before generating the stubs, be sure to install the `Hello.wsdl` file according to the instructions in Deploying the Service. To create the stubs, go to the *<JWSDP_HOME>*`/docs/tutorial/examples/jaxrpc/hello` directory and type the following:

```
ant generate-stubs
```

This command runs the `wscompile` tool as follows:

```
wscompile -gen:client -d build/client
-classpath build/shared config.xml
```

The `-gen:client` option instructs `wscompile` to generate client-side classes such as stubs. The `-d` option specifies the destination directory of the generated files.

The `wscompile` tool generates files based on the information it reads from the `Hello.wsdl` and `config.xml` files. The `Hello.wsdl` file was intalled on Tomcat when the service was deployed. The location of `Hello.wsdl` is specified by the `<wsdl>` element of the `config.xml` file, which follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration
  xmlns="http://java.sun.com/xml/ns/jax-rpc/ri/config">
  <wsdl location=
    "http://localhost:8080/hello-jaxrpc/hello?WSDL"
     packageName="hello"/>
```

```
</configuration>
```

The tasks performed by the `wscompile` tool depend on the contents of the `config.xml` file. For more information about the syntax of the `config.xml` file, see the XML Schema file: *<JWSDP_HOME>*`/docs/tutorial/examples/jaxrpc/common/jax-rpc-ri-config.xsd`.

Coding the Client

`HelloClient` is a stand-alone program that calls the `sayHello` method of the `HelloWorld` service. It makes this call through a stub, a local object which acts as a proxy for the remote service. Because the stubs is created before runtime (by `wscompile`), it is usually called a *static stub*.

To create the stub, `HelloClient` invokes a private method named `createProxy`. Note that the code in this method is implementation-specific and might not be portable because it relies on the `MyHello_Impl` object. (The `MyHello_Impl` class was generated by `wscompile` in the preceding section.) After it creates the stub, the client program casts the stub to the type `HelloIF`, the service definition interface.

The source code for `HelloClient` follows:

```
package hello;

import javax.xml.rpc.Stub;

public class HelloClient {
    public static void main(String[] args) {
        try {
            Stub stub = createProxy();
            HelloIF hello = (HelloIF)stub;
            System.out.println(hello.sayHello("Duke!"));
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    private static Stub createProxy() {
        // Note: MyHello_Impl is implementation-specific.
        return (Stub)(new MyHello_Impl().getHelloIFPort());
    }
}
```

Compiling the Client Code

Because the client code refers to the stub classes, be sure to follow the instructions in Generating the Stubs before compiling the client. To compile the client, go to the *<JWSDP_HOME>*/docs/tutorial/examples/jaxrpc/hello directory and type the following:

```
ant compile-client
```

Packaging the Client

To package the client into a JAR file, type the following command:

```
ant jar-client
```

This command creates the `dist/hello-client.jar` file.

Running the Client

To run the `HelloClient` program, type the following:

```
ant run
```

The program should display this line:

```
Hello Duke!
```

The `ant run` target executes this command:

```
java -classpath <cpath> hello.HelloClient
```

The classpath includes the `hello-client.jar` file that you created in the preceding section, as well as several JAR files that belong to the Java WSDP. In order to run the client remotely, all of these JAR files must reside on the remote client's computer.

Iterative Development

In order to show you each step of development, the previous sections instructed you to type several `ant` commands. However, it would be inconvenient to type all of those commands during iterative development. To save time, after you've initially deployed the service, you

can iterate through these steps:

1. Test the application.
2. Edit the source files.
3. Execute `ant build` to create the deployable WAR file.
4. Execute `ant redeploy` to undeploy and deploy the service.
5. Execute `ant build-static` to create the JAR file for a client with static stubs.
6. Execute `ant run`.

Implementation-Specific Features

To implement the JAX-RPC Specification, the Java WSDP requires some features that are not described in the specification. These features are specific to the Java WSDP and might not be compatible with implementations from other vendors. For JAX-RPC, the implementation-specific features of the Java WSDP follow:

- `config.xml` - See Generating the Stubs for an example.
- `jaxrpc-ri.xml` - See Packaging the WAR File for an example.
- ties - In the preceding example, the ties are in the `hello-jaxrpc.war` file, which is implementation-specific. (The `hello-portable.war` file, however, is not implementation-specific.)
- stubs - The stubs are in the `hello-client.jar` file. Note that the `HelloClient` program instantiates `MyHelloImpl`, a static stub class that is implementation-specific. Because they do not contain static stubs, dynamic clients do not have this limitation. For more information about dynamic clients, see the sections A Dynamic Proxy Client Example and A Dynamic Invocation Interface (DII) Client Example .
- tools - The `wsdeploy`, `wscompile`, and `deploytool` utilities.
- support for collections - See Table 9-1.



This tutorial contains information on the 1.0 version of the Java Web Services Developer Pack.

All of the material in *The Java Web Services Tutorial* is copyright-protected and may not be published in other works without express written permission from Sun Microsystems.